

Unit 1

1. Introduction to Artificial Intelligence

Artificial Intelligence (AI) is a field of computer science that focuses on creating systems capable of performing tasks that normally require human intelligence.

- Understanding language
- Recognizing images
- Making predictions
- Solving problems
- Understanding speech
- Making decisions
- Generating content

Examples of AI applications include voice assistants, recommendation systems, face recognition, fraud detection, chatbots, autonomous vehicles, and Generative AI systems.

A recommendation system on an online shopping platform can analyze a user's previous activity and recommend products that may be relevant to the user.

2. Introduction to Generative AI

Generative Artificial Intelligence (Generative AI) is a branch of AI that focuses on creating new content based on patterns learned from existing data.

- Text
- Images
- Audio
- Video
- Music
- Code
- Presentations
- Synthetic data

For example, a traditional AI system might determine whether an email is spam or not spam, while a Generative AI system can write a new email based on a user's instructions.

Basic workflow:

Training Data → Model learns patterns → Generative AI Model → User Prompt → Generated Content

Example prompt:

Write a professional email requesting two days of leave.

The AI can generate an appropriate email.

3. How Generative AI Works

Generative AI models are trained on large amounts of data. During training, the model learns patterns, relationships, structures, and characteristics within the data.

- Grammar
- Vocabulary
- Sentence structure
- Relationships between words
- Writing styles
- Programming syntax

After training, the model can generate new content based on an input prompt.

Prompt: Artificial Intelligence is
Possible continuation: a field of computer science...

By repeatedly predicting appropriate tokens, a language model can generate a complete response.

4. Generative AI vs Discriminative AI

Discriminative AI

A discriminative model focuses on distinguishing between categories or predicting an outcome.

Email → Spam Detection Model → Spam / Not Spam

Generative AI

A generative model learns patterns in data and generates new content.

Prompt: "A futuristic city with flying cars" → Generated Image

Feature	Generative AI	Discriminative AI
Main purpose	Generate new content	Classify or predict
Output	New content	Label/prediction
Example	Text generation	Spam detection
Image application	Generate an image	Identify whether an image contains a cat
Language application	Write an article	Classify sentiment
Main question	What can be generated?	Which category does this belong to?

5. Applications of Generative AI

5.1 Content Creation

Generative AI can create blog articles, social media posts, product descriptions, advertisements, emails, stories, and reports.

Write a 100-word product description for a wireless headphone.

5.2 Image Generation

A futuristic university campus surrounded by mountains.

Applications include advertising, graphic design, gaming, product design, education, and marketing.

5.3 Code Generation

- Writing code
- Explaining code
- Debugging
- Creating test cases
- Generating documentation
- Converting code between languages

Write a Python program to calculate the average of five numbers.

5.4 Business Automation

- Email drafting
- Report generation
- Document summarization
- Customer support
- Meeting summaries
- FAQ generation

5.5 Education

- Creating quizzes
- Generating practice questions
- Explaining difficult concepts
- Summarizing chapters
- Creating programming exercises
- Generating examples
- Creating study notes

6. Basics of Python Programming

Python is a high-level, interpreted, general-purpose programming language. It is widely used in Data Science, Machine Learning, Artificial Intelligence, Generative AI, Automation, and Web Development.

7. Python Variables

A variable is a name used to store a value.

```
name = "Rahul"  
age = 21  
marks = 87.5
```

Python automatically determines the type of the value.

```
x = 10  
print(type(x))
```

Output: <class 'int'>

Variable naming rules:

- Can contain letters, numbers, and underscore.
- Cannot start with a number.
- Cannot contain spaces.
- Is case-sensitive.
- Should have a meaningful name.

```
student_name = "Rahul"
student_marks = 85
```

8. Python Data Types

Data Type	Description	Example
Integer	Stores whole numbers.	age = 21
Float	Stores decimal numbers.	price = 499.99
String	Stores text.	course = "Generative AI"
Boolean	Stores True or False.	is_student = True
List	Ordered and mutable collection.	marks = [85, 90, 76, 88]
Tuple	Ordered collection that is generally immutable.	coordinates = (10, 20)
Dictionary	Stores key-value pairs.	student = {"name": "Rahul", "marks": 85}
Set	Stores unique values.	numbers = {10, 20, 10}

Example of accessing a list element:

```
marks = [85, 90, 76, 88]
print(marks[0])
```

Output: 85

Example of a dictionary:

```
student = {"name": "Rahul", "age": 21, "marks": 85}
print(student["name"])
```

Output: Rahul

9. Python Operators

```
a = 10
b = 3
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a % b)
```

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
**	Exponent
//	Floor division

Comparison operators return Boolean values such as True or False.

```
x = 10
y = 20
print(x > y)
print(x < y)
print(x == y)
```

10. Data type casting

Data type casting is the process of converting one data type into another.

1. Implicit Casting

- Conversion is done **automatically by Python**.
- Usually occurs between numeric data types.

```
x = 10
y = 2.5
z = x + y
print(z)    # 12.5
int → float
```

2. Explicit Casting

- Conversion is done **manually by the programmer**.
- Common functions: int(), float(), str(), bool().

```
x = "10"
y = int(x)
print(y)    # 10
```

Common Functions

Function	Purpose	Example
int()	Convert to integer	int("10")
float()	Convert to float	float("10.5")
str()	Convert to string	str(10)
bool()	Convert to Boolean	bool(1)

In short:

Implicit = Automatic

Explicit = Manual

Essential Python Libraries

NumPy → Numerical Computing
Pandas → Data Analysis
Matplotlib → Data Visualization
Seaborn → Statistical Visualization

12. NumPy

NumPy (Numerical Python) is a library used for numerical computing. It provides powerful arrays and mathematical operations.

```
import numpy as np

numbers = np.array([10, 20, 30, 40, 50])
print(numbers)

numbers = np.array([10, 20, 30])
print(numbers + 5)
```

Output: [15 25 35]

```
np.mean(numbers)
np.max(numbers)
np.min(numbers)
np.sum(numbers)
np.std(numbers)
```

These operations are useful for numerical analysis.

13. Pandas

Pandas is a Python library used for data manipulation and analysis. It is useful for CSV files, Excel data, tables, databases, student records, sales data, and business datasets.

```
import pandas as pd
```

A DataFrame is a two-dimensional table containing rows and columns.

```
data = {
    "Name": ["Rahul", "Aman", "Priya"],
    "Marks": [85, 92, 78]
}

df = pd.DataFrame(data)
print(df)

print(df["Marks"])
print(df["Marks"].mean())
print(df[df["Marks"] > 80])

df = pd.read_csv("students.csv")
```

Pandas can convert raw data into a structured form for analysis.

14. Matplotlib

Matplotlib is a Python library used to create graphs and charts such as line charts, bar charts, scatter plots, histograms, and pie charts.

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr"]
sales = [100, 150, 130, 180]

plt.plot(months, sales)
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Monthly Sales")
plt.show()
```

The graph can help a business understand changes in sales over time.

15. Seaborn

Seaborn is a Python visualization library built on top of Matplotlib. It is particularly useful for statistical data visualization.

```
import seaborn as sns
import matplotlib.pyplot as plt

marks = [65, 70, 75, 80, 85, 90]
sns.histplot(marks)
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.title("Distribution of Student Marks")
plt.show()
```

Common Seaborn functions include `histplot()`, `scatterplot()`, `boxplot()`, `barplot()`, `heatmap()`, and `countplot()`.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("students.csv")
sns.barplot(data=df, x="Name", y="Marks")
plt.show()
```

Pandas loads and prepares the data, Seaborn creates the statistical visualization, and Matplotlib displays or customizes the graph.

16. Introduction to Prompts

A prompt is an instruction or input provided to a Generative AI model. A clear prompt generally provides the model with better guidance.

Prompt → AI Model → Generated Output

Simple Prompt

Explain Python.

Specific Prompt

Explain Python programming to a beginner in 100 words.

Prompt with Context

I am teaching Python to first-year engineering students.
Explain variables using three simple programming examples.

Prompt with Role

Act as a Python instructor and explain lists to a beginner.

Prompt with Output Format

Explain Generative AI in a table with columns:
Definition, Applications, Advantages, and Limitations.

Prompt for Coding

Write a Python program to calculate the average of five numbers and explain the code.

Prompt for Summarization

Summarize this document in five bullet points. Use simple language and highlight the key concepts.

Prompt for Content Creation

Write a professional LinkedIn post about the importance of Generative AI for computer science students. Keep it within 150 words.

17. Prompt and Completion

A basic Generative AI interaction can be represented as:

Prompt → Model → Completion

The prompt is the input provided by the user. The completion is the output generated by the model.

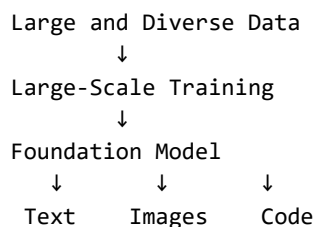
Prompt: Write a short description of Python.

Completion: Python is a high-level programming language known for its simple syntax and extensive libraries.

18. Foundation Models

A foundation model is a large AI model trained on broad and diverse datasets and designed to support many different tasks. The word foundation means that the model can serve as a base for developing many different AI applications.

Instead of creating a completely new AI model for every task, developers can use a foundation model and adapt it through prompting, fine-tuning, or other techniques.



Training Foundation Models

Depending on the model, training data may include books, websites, articles, code, images, audio, video, and other digital data.

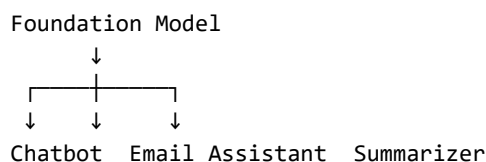
During training, the model learns patterns and relationships such as vocabulary, grammar, sentence structure, relationships between words, context, programming syntax, and writing patterns.

Examples of Foundation Models

- GPT
- Claude
- Gemini
- Llama
- Mistral

There are also foundation models designed for different modalities, including language models, vision models, and multimodal models.

Foundation Models in Applications



For example, a company can use a foundation model as the underlying capability for a customer-support chatbot, email assistant, document summarizer, question-answering system, and product-description generator.

Foundation Model vs Task-Specific Model

Feature	Foundation Model	Task-Specific Model
Purpose	Supports multiple tasks	Designed for a specific task
Training data	Broad and diverse	Usually task-specific
Reusability	High	More limited
Applications	Many	Narrower
Adaptation	Prompting, fine-tuning, etc.	Usually task-specific training
Example	General language model	Spam classifier

19. Large Language Models (LLMs)

A Large Language Model (LLM) is an AI model designed primarily to understand and generate human language. LLMs are an important category of Generative AI systems.

- Text generation
- Question answering
- Summarization
- Translation
- Classification
- Information extraction
- Code generation
- Conversational interaction
- Text rewriting
- Explanation

Examples of LLM Applications

Chatbot

What is Machine Learning?

Email Generation

Write a professional email requesting two days of leave.

Summarization

Summarize this 10-page document in five bullet points.

Translation

Translate "Good morning, how are you?" into Hindi.

Code Generation

Write a Python program to find the largest number in a list.

Code Explanation

Explain this Python code line by line.

Question Answering

What is the difference between supervised and unsupervised learning?

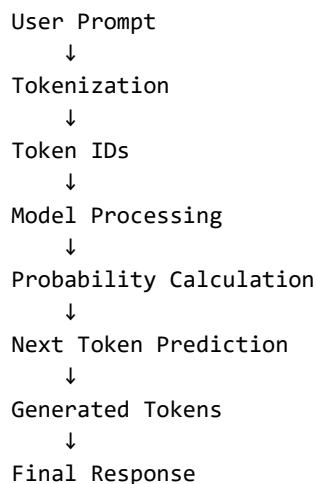
Data Analysis Assistance

Analyze these student marks and identify the highest and lowest scores.

Content Rewriting

Rewrite this paragraph in a professional tone.

How an LLM Processes a Prompt



The actual internal process is much more complex and involves neural-network architectures, attention mechanisms, learned parameters, and other components.

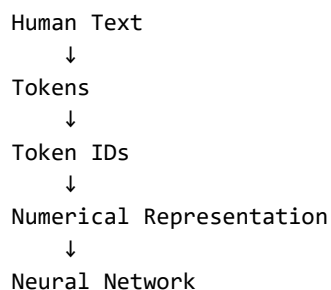
20. Tokenization

Tokenization is the process of breaking text into smaller units called tokens. Tokens can represent complete words, parts of words, punctuation, numbers, symbols, or other text units. The exact tokenization depends on the tokenizer used by a particular model.

Text: "I love Python."

Conceptual representation:
"I" | "love" | "Python" | "."

Why Tokenization Is Required



Language models operate on numerical representations rather than directly processing human-readable text.

Tokenization Example

Generative AI can create new content.

Conceptually:
Generative | AI | can | create | new | content | .

A tokenizer may also divide a longer or uncommon word into multiple tokens. For example, 'unhappiness' could conceptually be represented as 'un | happiness'. The exact representation depends on the tokenizer.

Tokens and Context Window

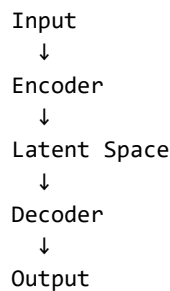
Language models process a limited amount of information within a context window. A conversation can contain a user question, previous conversation, documents, and generated responses, all of which can consume tokens.

More text → More tokens → More context consumed.

This is important when working with long documents, large prompts, long conversations, RAG systems, and large code files.

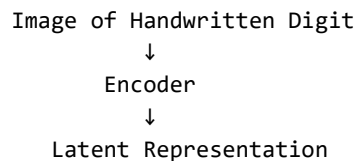
21. Variational Autoencoders (VAE)

A Variational Autoencoder (VAE) is a generative model that learns a compact representation of data and uses that representation to reconstruct or generate new data.



Encoder

The Encoder receives input data and converts it into a compact representation.

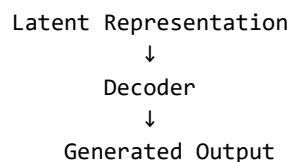


Latent Space

The latent space is a compact mathematical representation of important features of the data. For face images, it may capture characteristics related to face shape, expression, position, hair, and other visual characteristics.

Decoder

The Decoder takes information from the latent representation and generates an output.



VAE Example

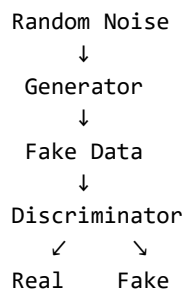
Suppose a VAE is trained using thousands of handwritten digit images. The model learns characteristics of different digits. After training, a new point can be sampled from the learned latent space, and the decoder can transform that representation into a new digit-like image.

Applications of VAE

- Image generation
- Data generation
- Image reconstruction
- Representation learning
- Anomaly detection
- Generating variations of data

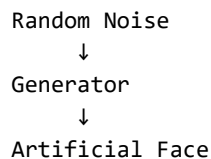
22. Generative Adversarial Networks (GANs)

A Generative Adversarial Network (GAN) contains two neural networks: Generator and Discriminator.



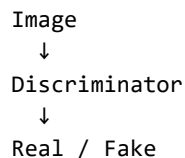
Generator

The Generator creates synthetic data. When trained on human faces, for example, it attempts to produce artificial faces.



Discriminator

The Discriminator tries to determine whether an input is real data from the training set or fake data produced by the Generator.



Generator and Discriminator Learning

The Generator tries to produce data that looks real, while the Discriminator tries to distinguish real data from generated data.

Generator → Fake Data → Discriminator → Feedback → Generator Improves

GAN Example

Suppose a GAN is trained on thousands of human-face photographs. Initially, the generator may produce poor-quality faces that are easy to identify as fake. As training progresses, it attempts to produce increasingly realistic samples.

GAN Applications

- Face generation
- Image synthesis
- Image enhancement
- Image-to-image translation
- Super-resolution
- Synthetic data generation
- Style transfer

23. Diffusion Models

Diffusion models are generative models that learn to generate data by reversing a gradual noise process. They are particularly important for modern image-generation systems.

The two main conceptual stages are forward diffusion and reverse diffusion.

Forward Diffusion

In the forward process, noise is gradually added to an original image.

Original Image
↓
Small Amount of Noise
↓
More Noise
↓
Heavy Noise
↓
Almost Random Noise

Reverse Diffusion

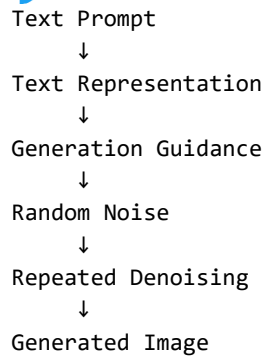
The model learns the reverse process. It starts from noisy data and repeatedly removes noise.

Random Noise
↓
Denoising
↓
More Structure
↓
Recognizable Content
↓
Generated Image

Text-to-Image Generation

Prompt:

"A student studying Artificial Intelligence in a modern university classroom."



The prompt provides information that guides the generation process.

Diffusion Example: Landscape Generation

A beautiful mountain landscape with a lake, green trees, and snow-covered peaks.

Diffusion Example: Product Design

A modern wireless headphone with a minimalist design displayed on a studio table.

Diffusion Example: Character Design

A futuristic robot character wearing an advanced protective suit, standing in a science-fiction city.

Diffusion Example: Background Replacement

Replace the background with a snowy mountain landscape.

A diffusion-based image-editing workflow can generate a new background while attempting to preserve relevant parts of the original image.

Diffusion Example: Image Restoration

Restore an old photograph by reducing noise and scratches while preserving important details.

Diffusion-based methods can be used as part of restoration workflows for noisy, scratched, or low-quality images.

Diffusion Example: Image Expansion

Extend the image to show more of the surrounding mountain landscape.

Generating additional content around an existing image is often called outpainting.

Diffusion Example: Image-to-Image Generation

Convert this photograph of a building into a watercolor-style illustration.

Diffusion Example: Advertisement Creation

A premium coffee cup placed on a wooden table, warm morning sunlight, coffee beans around the cup, professional advertising photography.

This can help a marketing team explore visual concepts for advertising.

Diffusion Example: Educational Illustration

An educational illustration showing the solar system with the planets arranged around the Sun.

Generated educational visuals should be checked for factual accuracy before instructional use.

Why Diffusion Models Are Powerful

- Text-to-image generation
- Image-to-image generation
- Image editing
- Image restoration
- Image expansion
- Creative design
- Concept generation
- Video-generation research

24. VAE vs GAN vs Diffusion Models

Feature	VAE	GAN	Diffusion Model
Main architecture	Encoder + Decoder	Generator + Discriminator	Denoising process
Main concept	Latent representation	Adversarial competition	Noise removal
Generation starting point	Latent representation	Random noise	Random noise
Training idea	Reconstruction + latent-space learning	Generator competes with discriminator	Learn reverse diffusion
Common use	Representation and generation	Synthetic image generation	Image generation and editing
Generation process	Decode latent representation	Generator creates sample	Iterative denoising

25. Simple Example Comparing VAE, GAN and Diffusion

VAE

Digit Images → Encoder → Latent Space → Decoder → New Digit

The VAE learns a compact representation of the digits.

GAN

Random Noise → Generator → Fake Digit → Discriminator → Real / Fake

The Generator learns to create increasingly realistic digits.

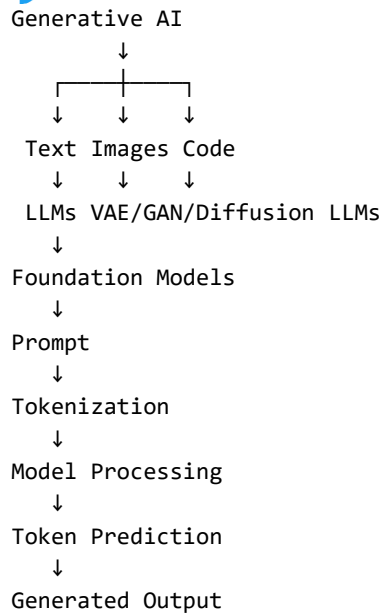
Diffusion

Random Noise → Denoising → Denoising → Denoising → Generated Digit

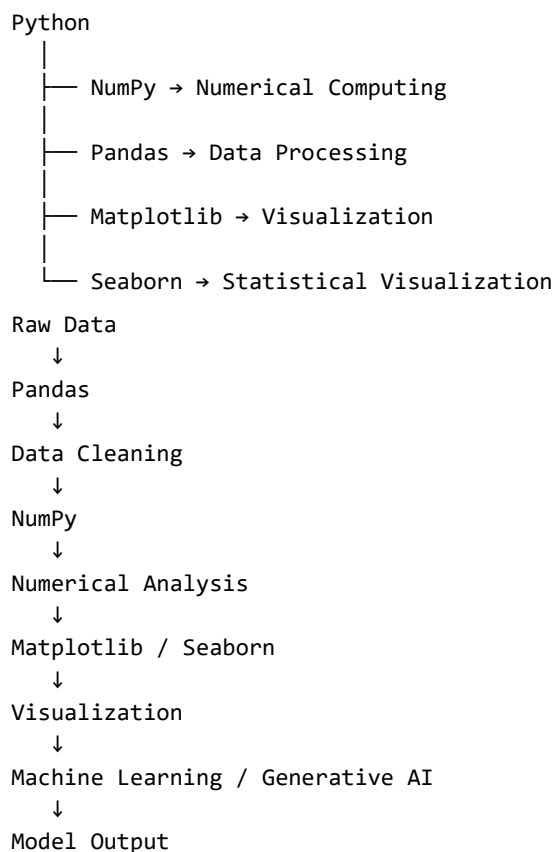
The model learns to transform noise into a meaningful sample through repeated denoising.

26. Overall Generative AI Ecosystem

Artificial Intelligence
↓
Machine Learning
↓



27. Role of Python in Generative AI



28. Key Points to Remember

1. Artificial Intelligence enables machines to perform tasks associated with human intelligence.
2. Generative AI focuses on creating new content.
3. Discriminative AI focuses on classification or prediction.

4. Generative AI can create text, images, audio, video, and code.
5. A prompt is an instruction provided to a Generative AI model.
6. A completion is the generated response.
7. A foundation model is a broad, reusable AI model trained on large and diverse datasets.
8. LLMs are language-focused models that can understand and generate text.
9. Tokenization converts text into smaller units called tokens.
10. Token IDs provide a numerical representation that a model can process.
11. NumPy is used for numerical computing.
12. Pandas is used for data manipulation and analysis.
13. Matplotlib is used for visualization.
14. Seaborn is used for statistical visualization.
15. VAE uses an encoder, latent representation, and decoder.
16. GAN consists of a Generator and Discriminator.
17. Diffusion models learn to generate data through a process involving noise and learned denoising.
18. Diffusion models are used for image generation, editing, restoration, and expansion.
19. Foundation models and LLMs can support many different AI applications.
20. Python and its data-science libraries provide an important foundation for AI development.